

Technologie Informacyjne i Komunikacyjne R

Linux – skrypty

Katarzyna Grzelak

Zakład Częstek i Oddziaływań Fundamentalnych
Uniwersytet Warszawski

Skrypty powłoki

- Skrypt powłoki = plik tekstowy zawierający jedno lub wiele poleceń Linux'owych ...
- ... i polecenia sterujące (instrukcje warunkowe, pętle).
- Skryptowy język programowania (!)

Rozruch powłoki, standardowo uruchamiane skrypty

- **Jeśli bash jest powłoką logowania**
(np. `ssh user@host`)
 - `/etc/profile`
 - `~/.bash_profile`
 - ew. `~/.bash_login`
 - ew. `~/.profile`

Rozruch powłoki, standardowo uruchamiane skrypty

- Powłoka potomna w trybie interakcyjnym:
`~/ .bashrc`
- Powłoka potomna nie w trybie interakcyjnym:
plik zdefiniowany w zmiennej środowiska
`$BASH_ENV`

Zamykanie powłoki, standardowo uruchamiany skrypt
`~/ .bash_logout`

- Powłoka zakłada że każda linia w skrypcie to osobne polecenie
- Komentarze zaczynają się od znaku #
- Jeżeli pierwszą linijką w skrypcie jest:
`#!/bin/bash`
to dalsze polecenia będą realizowane w powłoce `bash`, niezależnie od powłoki której używamy w terminalu z którego wywołujemy skrypt

Różne sposoby uruchamiania skryptów

- `bash skrypt`
- Uruchomienie skryptu jak zwykłego programu:
`./skrypt`
jeżeli skrypt ma dodane prawa do wykonywania:
`chmod +x skrypt`
- `source skrypt`, `. skrypt` (nie jest uruchamiana powłoka potomna)

Różne sposoby uruchamiania skryptów

Jeżeli uruchamiamy skrypt poleceniem typu `source skrypt` (nie jest uruchamiana powłoka potomna) to wszystkie ustawienia z wnętrza skryptu będą obowiązywały w oknie terminala po zakończeniu wykonywania skryptu.

Przykład

- 1 Skrypt `skrypt1.sh` uruchamiamy z katalogu domowego:
`source skrypt1.sh`
- 2 Wewnątrz skryptu znajduje się polecenie
`cd ~/katalog`
- 3 Po zakończeniu działania skryptu, bieżący katalog w oknie terminala to `~/katalog`

Argumenty wiersza poleceń

Argumenty wiersza poleceń przekazywane są do skryptu i umieszczane w specjalnych zmiennych powłoki:

- \$0 – nazwa skryptu
- \$1 – pierwszy argument
- \$2 – drugi argument
- ...
- \$9 – dziewiąty argument
- \$# – liczba argumentów
- \$* - wszystkie argumenty

Przykład

- `./skrypt arg1 arg2 arg3 arg4`

Wypisywanie tekstu ze środka skryptu

Przykłady

```
echo "Z nowym wierszem"
```

```
echo -n "Bez nowego wiersza"
```

```
echo $0
```

```
echo $*
```

```
echo $HOSTNAME
```

Przykład: dopisanie nowej linii do pliku tekstowego (`.bashrc`), bez użycia edytora tekstu. Ciąg liter **KONIEC** można zastąpić dowolnym innym.

```
cat << KONIEC >> .bashrc  
echo Witam  
KONIEC
```

Zamykający ciąg liter musi być zapisany od początku nowej linii.

Sterowanie przebiegiem skryptów

```
licznik=60  
zmienna=ABC
```

Odwoływanie się do zmiennej

`$zmienna` vs `${zmienna}` (druga wersja dobrze definiuje koniec nazwy zmiennej)

Pętle w bash

- ```
for zmienna in lista
do
 polecenie_1
 polecenie_2
 ...
 polecenie_n
done
```
- ```
for nazwa_pliku in wzorzec
do
    polecenie_1
    polecenie_2
    ...
    polecenie_n
done
```

Pętle w bash - przykłady

- ```
for i in 1 3 5 7 9
do
 echo $i
done
```
- Można też użyć polecenia `seq`, np.:  

```
for i in `seq 1 10`
...

```
- ```
for plik in *
do
    echo $plik
done
```

Pętla w bash jak w języku C/C++

- max=*indeks_max*
for ((i=1; i <= max; i++))
do
 polecenie_1
 polecenie_2
 ...
 polecenie_n
done

- Przykład:
max=15
for ((i=1; i <= max; i++))
do
 echo \$i
done

Argumenty wiersza poleceń przy użyciu pętli for

Wypisywanie wszystkich argumentów \$1, \$2, \$3, ...

```
for i
do
    echo $i
done
```

Instrukcje warunkowe

Instrukcja warunkowa jest przystosowana do wykorzystywania wyników poleceń (warunek w nawiasie).

```
● if  (warunek) then
    polecenie_1
    ...
fi
● if  (warunek) then
    polecenie_1
    ...
else
    polecenie_2
    ...
fi
```

Kod powodzenia programu 0 → prawda

Kod niepowodzenia programu – wartość niezerowa → fałsz

Instrukcje warunkowe

- ```
if (warunek1) then
 polecenie_1
 ...
elif (warunek2) then
 polecenie_2
 ...
else
 polecenie_3
 ...
fi
```

Wyrażenie testujące, przykład:

```
test $x -eq $y
```

```
x=15
```

```
y=17
```

```
if (test $x -eq $y) then
```

```
 echo "x = y"
```

```
else
```

```
 echo "x != y"
```

```
fi
```

# Porównywanie wartości liczbowych

`$x -eq $y`

`$x -ne $y`

`$x -gt $y`

`$x -ge $y`

`$x -lt $y`

`$x -le $y`

# Porównywanie ciągów tekstowych

Wyrażenia do porównywania ciągów tekstowych:

- `"$c1" = "$c2"`      ciągi są jednakowe
- `"$c1" != "$c2"`      ciągi nie są jednakowe
- `"$c1"`      ciąg jest niepusty
- `-z "$c1"`      długość ciągu jest zerowa
- `-n "$c1"`      długość ciągu jest niezerowa

Cudzysłowy ważne, jeżeli ciąg ze spacjami.

Przykład:

```
napis="Ala ma kota"
if (test "$napis" = "Ala ma kota") then
 echo "Napisy są takie same"
else
 echo "Napisy są różne"
fi
```

# Opcje polecenia test dla plików

|                               |                                 |
|-------------------------------|---------------------------------|
| <code>-e nazwa_pliku</code>   | plik (lub katalog) istnieje     |
| <code>! -e nazwa_pliku</code> | plik (lub katalog) nie istnieje |
| <code>-d nazwa_pliku</code>   | plik jest katalogiem            |
| <code>-f nazwa_pliku</code>   | plik jest zwykłym plikiem       |
| <code>-s nazwa_pliku</code>   | plik ma niezerowy rozmiar       |
| <code>-r nazwa_pliku</code>   | plik można odczytać             |
| <code>-w nazwa_pliku</code>   | plik można modyfikować          |
| <code>-x nazwa_pliku</code>   | plik wykonywalny                |

# Instrukcje warunkowe - skrócony zapis

- ```
if ( [ wyrażenie testujące ] ) then
    polecenie_1
    ...
else
    polecenie_2
    ...
fi
```
- ```
if [wyrażenie testujące]
then
 polecenie_1
 ...
else
 polecenie_2
 ...
fi
```

Spacje są ważne !

## Przykład testu dla plików, skrócony zapis instrukcji warunkowej

```
if [-e config.txt]
then
 echo "Plik config.txt istnieje"
fi
```

# Operatory logiczne

- & &      operacja logiczna AND
- ||      operacja logiczna OR
- !      negacja

Użycie np.

```
if (test -z $1 || test -z $2) then
```

```
...
```

# Zmienne powłoki, zmienne środowiska

# Zmienne środowiska, zmienne powłoki

- Zmienne środowiska (*environment variables*) – globalne zmienne definiujące środowisko pracy, ustawiane na ogół w plikach konfiguracyjnych i dziedziczone przez wszystkie egzemplarze powłoki
- Zmienne powłoki (*shell variables*) – definiowane w danej powłoce, lokalne względem danego egzemplarza powłoki
- To co się kryje pod daną sekwencją liter można odczytać używając polecenia `echo` i symbolu `$` przed nazwą, np.:

```
echo $PATH
```

```
echo $HOSTNAME
```

```
echo $SHELL
```

# Zmienne środowiska vs zmienne powłoki (bash)

`set` - wypisanie wszystkich zmiennych

`printenv` - wypisanie tylko zmiennych środowiska

- Przykłady definicji zmiennych:

```
HISTSIZE=60
```

```
NAPIS="Bardzo dlugi napis"
```

- Odwoływanie się do zmiennych:

```
$HISTSIZE, $NAPIS
```

- Zwalnianie nazw zmiennych:

```
unset NAPIS
```

# Ustawianie zmiennych środowiska (bash)

```
ZMIENNA=WARTOSC
export ZMIENNA
lub
export ZMIENNA=WARTOSC
set -a - eksport wszystkich zmiennych powłoki
```

Przy definiowaniu zmiennych ważne są spacje (lub ich brak).

- Powłoka wyświetla znak zachęty (zwykle \$, lub %), który sygnalizuje jej gotowość do przyjmowania poleceń
- Przykładowa modyfikacja znaku zachęty w bash:  
`PS1="$ {USER}@${HOSTNAME} : \w$ "`

# Po co różne apostrofy ?

- Polecenie `echo pwd` powoduje wypisanie na ekran słowa `pwd`
- Polecenie `echo `pwd`` (lub `echo $(pwd)`) powoduje uruchomienie polecenia `pwd` i wypisanie wyniku tego polecenia na ekran
- Polecenie `echo "$PATH"` (równoważne `echo $PATH`) wypisuje ścieżkę dostępu (`$` jest traktowany jako znak specjalny)
- Polecenie `echo '$PATH'` wypisuje `$PATH` (`$` jest traktowany dosłownie)